# dinosn/cve-2026-103849-blockstrap-lab: README.md

**dinosn/cve-2026-103849-blockstrap-lab: README.md** - Author not stated, GitHub.

- Published: date not stated
- Original: <https://github.com/dinosn/cve-2026-103849-blockstrap-lab/blob/e9ce56baa725bcfdd28570156c0cff1a4203a1e6/README.md>
- Preserved from: https://github.com/dinosn/cve-2026-103849-blockstrap-lab/blob/e9ce56baa725bcfdd28570156c0cff1a4203a1e6/README.md (github-api) on 2026-10-21
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

## Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

> **Repository reading copy.** Created from README.md.

# README.md

`dinosn/cve-2026-103849-blockstrap-lab` at `e9ce56baa725`, path `README.md`.

# CVE-2026-103849 / BlockStrap validation lab

[[image: Static validation]](https://github.com/dinosn/cve-2026-103849-blockstrap-lab/actions/workflows/static-validation.yml)

> [!IMPORTANT] This repository proves the stock BlockStrap directive-injection primitive, an unauthenticated HTTP carrier, and a bounded stock file side effect. The command-execution track requires the clearly labeled synthetic lab gadget; it does not claim a stock Magento full-RCE chain.

This lab reproduces the core second-pass `{{block}}` directive-injection primitive fixed by Adobe in APSB26-153, including a stock unauthenticated HTTP carrier. It uses Magento Open Source 2.4.9 at commit `755e34dd689021c5165db9d35ecff74f7dc51527` and keeps the September VULN-39341 StyleSmuggler hotfix installed in both test states.

The current claim-by-claim audit is in REANALYSIS.md.

## Safety and requirements

Run the lab only on a system you control. The exposed HTTP and MailHog ports bind to `127.0.0.1`; database, Redis, OpenSearch, PHP-FPM, and SMTP remain on the private Compose network. The command demonstration accepts only a random nonce and executes the literal command `id 2>&1`; it cannot accept shell text, an output path, or file content from the operator.

The validated environment used Docker Engine with Compose v2 on `amd64`. Every base and service image is pinned to an immutable registry digest; the MailHog image is specifically pinned to its `linux/amd64` manifest. Host-side checks and wrappers require Bash, GNU Make, Python 3.10 or newer, PHP CLI, and standard Unix utilities including `awk`, `comm`, `find`, and either `sha256sum` or `shasum`. PHP CLI is used only by `make verify`; Magento itself runs in Docker. Allow at least 8 GB of memory, 10–15 GB of free disk, and internet access during the initial image and Magento build. Other architectures may work but are not covered by the recorded evidence.

On Linux, OpenSearch requires `vm.max_map_count` of at least `262144`. For example, set it for the current boot with `sudo sysctl -w vm.max_map_count=262144`. If `10.210.103.0/24` overlaps a local network, change the subnet under `networks.default.ipam` in `compose.yaml` before startup.

The credentials in `.env.example` are isolated lab defaults. Do not reuse them for any other system.

The stock primitive probe is deliberately non-destructive. It asks Magento's stock `Magento\\Framework\\View\\Element\\Text` block to run `getCacheKey()` over a fresh nonce. The exact success value, `CUSTOM_BLOCK_<nonce>`, never appears in the input. That probe does not execute a command, write a file, make a callback, or accept an operator-supplied payload. Separate opt-in probes cover a bounded stock file side effect and a lab-only conditional command-execution case.

## Proof boundary

The lab has four evidence tracks: a component A/B, a stock unauthenticated HTTP A/B, a bounded stock file-side-effect A/B, and a conditional command-execution A/B using the explicitly installed lab gadget described below.

### Component proof

The A/B test proves the BlockStrap template-engine primitive:

```
root email filter sees {{template ...}}
  -> callback renders attacker-shaped data through a child framework filter
  -> child cannot process {{block ...}} and returns it unchanged
  -> vulnerable engine wraps that attacker-controlled directive in its signature
  -> root engine generically rescans the signed text
  -> root email filter invokes blockDirective()
  -> stock Text block transforms a fresh nonce with its CUSTOM_BLOCK_ prefix
```

With the October fix, the child output is not generically signed or rescanned. Only directives explicitly placed in the new `DeferredDirectiveRegistry` can be resolved by the root, so the identical `{{block}}` remains inert.

This proof uses the real Magento object manager, directive processors, shared filtering-depth meter, signature provider, layout, and block factory.

## Unauthenticated HTTP proof

The HTTP probe uses only public guest GraphQL operations and a local SMTP sink:

```
createEmptyCart
  -> setGuestEmailOnCart
  -> setBillingAddressOnCart with the recursive address-format carrier
  -> handlePayflowProResponse with a declined response
  -> recursive address formatting reaches a nested base-filter pass
  -> its signed output is stored as billingAddressHtml
  -> a later email-filter pass shares the request signature and rescans it
  -> vulnerable only: stock Text::getCacheKey() returns CUSTOM_BLOCK_<nonce>
  -> MailHog captures the transformed value
```

Both states must capture exactly one failed-payment email correlated by the fresh nonce. The fresh success value is absent from every HTTP request and appears only when `blockDirective()` invokes the stock block. In the patched state an equivalently generated email arrives without that value. This proves a working unauthenticated request-to-block-execution path.

This stock HTTP probe does not establish operating-system command execution. It contains no command, PHP expression, file-write gadget, callback, or operator payload. It is not presented as one of Sansec's four withheld HTTP-to-RCE chains. No executable public full-RCE PoC was identified in the bounded search performed on 2026-10-18. The search record is retained in evidence/2026-10-18-public-poc-search.md.

## Conditional command-execution demonstration (lab-only gadget)

`poc/http_rce_probe.py` plus the `BlockStrapLab_Gadget` module (`gadget/`) demonstrates what happens when a deployment contains a compatible block gadget. The module is authored for this lab and deliberately executes the fixed harmless command `id`; it is not a

Magento stock class or a recovered Sansec chain. Every preceding link stays stock: guest GraphQL carrier, nested-filter signature wrapping, root `blockDirective()` dispatch, and zero-argument `output=` method selection.

No compatible command-execution gadget was found in the reviewed subset of this exact Magento Open Source 2.4.9 tree. That negative search is incomplete: the dynamic sweep excluded some namespaces and constructor shapes, exercised only selected methods, and retained no packaged raw run. It does not prove absence across Magento Open Source, Adobe Commerce, extensions, versions, or deployment configurations. Stock full RCE therefore remains unproven here.

```
unauthenticated guest GraphQL flow
  -> recursive billing-address carrier stored via setBillingAddressOnCart
  -> handlePayflowProResponse decline -> PaymentFailuresService email
  -> nested base-filter pass leaves {{block}} unchanged and signs it
  -> later email filter rescans and dispatches blockDirective()
  -> lab gadget runs fixed `id` as www-data
  -> nonce-correlated, base64-wrapped uid output in MailHog + marker file
```

Run:

```
make http-rce-ab
```

`make http-rce-ab` runs a self-contained A/B wrapper that establishes its cleanup trap before installing the gadget, then removes it at exit. The patched state must still capture the correlated failed-payment email while showing neither the gadget marker nor a marker file. Recorded run: VALIDATION.md and evidence/2026-10-18-rce-ab.txt.

### Bounded server-side side effect

A separate probe replaces the Text block with the stock `Magento\Backend\Block\Widget\Grid\Extended` block and selects its public `getCsvFile()` method. On this exact source tree, that method creates a random `var/export/<32-hex>.csv` containing the fixed four bytes `EF BB BF 0A` before it fails because the directive-created grid has no collection.

The host wrapper snapshots `var/export` before and after the request. The vulnerable state must create exactly one file with the required name pattern, size, and content shape; the patched state must create none. The wrapper deletes only the newly observed matching file and restores the patched source in a trap.

This proves a persistent server-side side effect from the unauthenticated request. The attacker cannot select the path or content, and it is not an arbitrary file write or RCE.

## Pinned states

| State | VULN-39341 | APSB26-153 BlockStrap engine fix | Expected result |
|---|---|---|---|
| `vulnerable` | applied | absent | child wraps the directive; root executes Text block |
| `patched` | applied | applied | directive remains inert |

Patch artifacts are checksum-pinned:

- `VULN-39341_249-monorepo.patch`: the prior StyleSmuggler hotfix, mapped to

Magento's public monorepo paths.

- `APSB26-153_BlockStrap_249-monorepo.patch`: the four BlockStrap engine files

extracted from Adobe's complete October patch and path-mapped for the public monorepo.

The state switcher refuses to modify a source tree that matches neither known patch state. NOTICE.md records the official source archives, hashes, transformations, and upstream licensing. The complete October patch is not redistributed. After downloading both official archives, reproduce and compare the shipped patches byte-for-byte with:

```
python3 scripts/verify-patch-provenance.py \
  --vuln-archive /path/to/VULN-39341-composer-patches.zip \
  --october-archive /path/to/2-4-9-oct-2026.zip
```

## Run

The Magento gateway and MailHog API bind only to loopback. Compose uses the dedicated `10.210.103.0/24` bridge because the validation host's automatic Docker address pools are already allocated.

```
cp .env.example .env
make full
```

`make full` runs the static verifier, builds and waits for the stack, then executes all four vulnerable/patched A/B tracks in sequence.

Useful commands:

```
make status       # show this lab's containers
make full         # build and run the complete validation sequence
make vulnerable   # VULN-39341 only
make patched      # VULN-39341 plus the October fix
make probe        # probe the currently selected state
make http-ab       # test the unauthenticated HTTP/email path in both states
make file-ab       # test and clean the bounded var/export side effect
make http-rce-ab   # conditional fixed-command A/B; removes lab gadget
make gadget-remove # remove/disable the lab gadget without running the probe
make logs         # follow first-boot output
make down         # stop and preserve the lab volumes
make reset        # delete only this lab's containers, network, and volumes
```

Settings copied from `.env` during first installation require `make reset` before they take effect again. That command deletes this lab's database, OpenSearch index, Magento source, and other named volumes.

All A/B wrappers verify that the source is patched before reporting success. The file wrapper also verifies removal of every matching CSV created since its initial snapshot. The conditional command wrapper verifies marker removal and that the synthetic gadget is disabled and removed. `make ab` passes only when:

- vulnerable: child output was textually wrapped and the stock block returned

the fresh transformed value that was absent from the input;

- patched: child output was not wrapped, the block did not execute, and the

original directive remained literal;

- both runs used the same Magento revision, September hotfix, probe, and nonce

generation logic.

`make http-ab` passes only when:

- both runs create a guest cart, store the nested address, trigger the same

failure handler, and capture an email;

- vulnerable: the captured email contains `CUSTOM_BLOCK_<fresh nonce>`;
- patched: an email is still captured but the transformed marker is absent;
- the transformed marker is absent from all requests in both states.

`make file-ab` passes only when:

- vulnerable: exactly one new `var/export/<32-hex>.csv` exists, is four bytes,

and has SHA-256 `b42f2099187886def637d6aa840022266e05cb6c987a9394e708e23cd505eb46`;

- patched: the same HTTP route completes without creating a new CSV;
- cleanup removes only matching fixed-format files created after the initial

snapshot, and the source is verified in the patched state.

See VALIDATION.md for the sanitized recorded run and evidence/README.md for the public-evidence policy.

## Sources

- Adobe APSB26-153: <https://www.adobe.com/trust/security/products/commerce/apsb26-153.html>
- Sansec BlockStrap analysis: <https://sansec.io/research/blockstrap-magento-rce>
- Mage-OS public fix port: <https://github.com/mage-os/mageos-magento2/pull/365>

Archived from https://github.com/dinosn/cve-2026-103849-blockstrap-lab/blob/e9ce56baa725bcfdd28570156c0cff1a4203a1e6/README.md. Rendered offline from the local Markdown copy.